

FIT3155 S1/2026: Assignment 2

(Due date: 11:55pm Friday 29 May 2026)

[Weight: 20 = 10 + 10 marks.]

Assessment guidelines:

- You **must write all code yourself**. Use of Generative AI tools is **not** allowed.
- Any code **derived from online sources must be declared and fully cited**.
- Assessment is about your **understanding and ability to write code yourself**.
- You may be **called for an interview** to justify your understanding of your code.
- You must not use **external library routines**, except those considered standard or explicitly allowed in the specification below.
 - Standard data structures (such as lists, dictionaries, tuples, and sets) are allowed if they do not conflict with the assessment objectives. Ensure their use is appropriate and efficient in terms of space and time.

Submission instructions:

- All your scripts **must** contain your name and student ID.
- Upload a **.zip** archive via Moodle with the filename of the form **a2_<studentID>.zip**.
 - Your archive should extract to a directory named as your Monash student ID.
 - This directory should contain a subdirectory for each of the questions, named as: **q1/** and **q2/**.
 - The corresponding scripts and other question-specific reporting material (as stated below) should be placed in these subdirectories.

Academic integrity

Monash University's academic integrity policy is accessible via this link: <https://www.monash.edu/students/academic/policies/academic-integrity> (click). Read it carefully to understand your responsibilities. **As per FIT policy, all submissions will be scanned via MOSS (click) and/or JPlag (click).**

Assignment Questions

1. Constructing Longest Common Prefix (LCP) array using suffix **tree**.

Background: You have seen in Week 3 that, given a string $S[1 \dots n] \$$, the suffix array $SA[1 \dots N]$ is an array of starting positions of all suffixes of $S \$$ in their lexicographic order. An $LCP[1 \dots N]$ array, on the other hand, is an array where any $LCP[i]$ stores the **length of the longest common prefix** between the suffixes $S[SA[i-1] \dots n \$]$ and $S[SA[i] \dots n \$]$. (Note, by convention, $LCP[1] = 0$.)

- **Example:** Let $S = \text{googol} \$$. The sorted suffixes and corresponding LCP values are:

Suffix Array	LCP Values
$SA[1] = 7$ (\$)	$LCP[1] = 0$ (by convention)
$SA[2] = 4$ (gol\$)	$LCP[2] = 0$ (\$, gol\$)
$SA[3] = 1$ (googol\$)	$LCP[3] = 2$ (gol\$, googol\$)
$SA[4] = 6$ (l\$)	$LCP[4] = 0$ (googol\$, l\$)
$SA[5] = 3$ (ogol\$)	$LCP[5] = 0$ (l\$, ogol\$)
$SA[6] = 5$ (ol\$)	$LCP[6] = 1$ (ogol\$, ol\$)
$SA[7] = 2$ (oogol\$)	$LCP[7] = 1$ (ol\$, oogol\$)

Your assignment task: You must write an efficient Python program to compute the LCP array of any input string $S[1 \dots n]$. For this task, assume that the input string $S[1 \dots n]$ consists of characters in the ASCII range [37, 126]. After reading the input from the file, your program must concatenate the terminal symbol \$ to S before proceeding.

In your attempt, **you must strictly use a suffix tree** and then exploit its structure to compute the LCP values. No third-party algorithm for computing the LCP array from a suffix array is allowed, and your solution must rely on the constructed suffix tree directly.

As a requirement of this task, you must implement Ukkonen's algorithm taught in Week 4. You must not rely on third-party implementations or external libraries, as the goal is to demonstrate your understanding of both the algorithm and the suffix tree data structure being assessed in this task. As part of the construction, you are required to produce a run log (see the run log specification provided below, with an example).

Strictly follow the following specification to address this question:

Program name: a2q1.py

Argument to your program: A single input filename of a file containing the string $S[1 \dots n]$ as one line.

(Terminal \$ will not be included in this file. Your program has to concatenate it to S .)

Command line usage of your script:

python a2q1.py <input filename>

Do not hard-code the input filename within your program. Ensure your program can be executed from the terminal using any valid input file supplied as a command-line argument. Include sufficient inline comments explaining each

*Since the special terminal \$ is appended to the input string $S[1 \dots n]$, denote $N = n + 1$.

logical block of your code. Code that lacks adequate comments will require a face-to-face interview to verify your understanding before it can be assessed.

Output file name: `output_a2q1.txt`

- The output must contain the LCP array of $S[1 \dots n]$, each $\text{LCP}[i]$ per line $\forall 1 \leq i \leq N$.

Example: For $S[1 \dots n] = \text{googol}$, the output file would contain:

```
0
0
2
0
0
1
1
```

No further test cases will be provided. You must develop your own test cases.

Run log filename: `runlog_a2q1.txt`

Note: The suffix tree construction phase of your program using Ukkonen's algorithm must generate a run log text file that records key information, in the format shown below for the example string `googol`. Specifically, the log must include the phase number, the suffix extension number (for explicit extensions), the numeric index of the active node,[†] the current remainder substring range, the suffix link of the active node, and the suffix link of any newly created internal node (which is to be reported when it is resolved in a subsequent extension). This information must be automatically produced by your program in the format shown in the example. (Note that, in the required format, an indent corresponds to four spaces. By convention, the root node is Node 1, and its suffix link points to itself.)

For $S[1 \dots n] = \text{googol}$, the run log would be:

```
Root Node 1

Phase 1 starts from Extn 1
  Extn 1 applies Rule 2 (alternate)
  Active Node = Node 1 (suffix link to Node 1); Remainder = EMPTY
  Node 2 created: Leaf node!

Phase 2 starts from Extn 2
  Extn 2 applies Rule 2 (alternate)
  Active Node = Node 1 (suffix link to Node 1); Remainder = EMPTY
  Node 3 created: Leaf node!

Phase 3 starts from Extn 3
  Extn 3 applies Rule 3
  Active Node = Node 1 (suffix link to Node 1); Remainder = EMPTY

Phase 4 starts from Extn 3
  Extn 3 applies Rule 2 (regular)
  Active Node = Node 1 (suffix link to Node 1); Remainder = S[3...3]
  Node 4 created: Internal node!
  Node 5 created: Leaf node!
  Extn 4 applies Rule 3
  Active Node = Node 1 (suffix link to Node 1); Remainder = EMPTY
```

[†]Each node created (internal or leaf) must be assigned a numeric index in the order of its creation during Ukkonen's algorithm, with the root node designated as Node 1 and other Nodes numbered 2, 3, etc. in the order they are created.

Linking Node 4 to Node 1 Phase 5 starts from Extn 4 Extn 4 applies Rule 3 Active Node = Node 1 (suffix link to Node 1); Remainder = S[4...4] Phase 6 starts from Extn 4 Extn 4 applies Rule 2 (regular) Active Node = Node 1 (suffix link to Node 1); Remainder = S[4...5] Node 6 created: Internal node! Node 7 created: Leaf node! Extn 5 applies Rule 2 (alternate) Active Node = Node 4 (suffix link to Node 1); Remainder = EMPTY Node 8 created: Leaf node! Linking Node 6 to Node 4 Extn 6 applies Rule 2 (alternate) Active Node = Node 1 (suffix link to Node 1); Remainder = EMPTY Node 9 created: Leaf node! Phase 7 starts from Extn 7 Extn 7 applies Rule 2 (alternate) Active Node = Node 1 (suffix link to Node 1); Remainder = EMPTY Node 10 created: Leaf node!
--

PDF Report: report_a2q1.pdf

- At a high-level, describe the logical approach you have taken to compute the LCP array using the suffix tree, while reflecting on its worst-case space and time requirement (including construction of the suffix tree).

2. Semi-numerical pattern matching.

Background:

- ***k*-gram:** A *k*-gram over an alphabet Σ is any string $S \in \Sigma^k$ of the form $s_1s_2 \dots s_k$ such that $s_i \in \Sigma$ for all $1 \leq i \leq k$. For example, if $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \dots, \mathbf{z}\}$ and $k = 3$, then **aaa**, **zwp**, **cpp**, and **myv** are a few 3-grams (out of the 26^3 possibilities for this alphabet).
- **Mapping a *k*-gram to an integer (modulo a prime number):** Any *k*-gram $S = s_1s_2 \dots s_k$ can be interpreted as a base- β number that can be mapped to an integer in $\mathbb{Z}_{\geq 0}$ as:

$$z(S) = \sum_{i=1}^k \text{index}(s_i) \times \beta^{k-i}$$

where $\text{index}(s_i)$ denotes the index of s_i in the alphabet Σ , counting from 0, and $\beta = |\Sigma|$ is the size of the alphabet from which S was generated. As an example, if $\Sigma = \{\mathbf{a}, \mathbf{b}, \mathbf{c}, \dots, \mathbf{z}\}$ (so $\beta = 26$), then the 6-gram $S = \mathbf{yzzyxz} \in \Sigma^6$ maps to the integer 297033671 as follows:

$$\begin{aligned}
z(S) &= \text{index}(\mathbf{y}) \times \beta^5 + \text{index}(\mathbf{z}) \times \beta^4 + \text{index}(\mathbf{z}) \times \beta^3 + \text{index}(\mathbf{y}) \times \beta^2 + \text{index}(\mathbf{x}) \times \beta^1 + \text{index}(\mathbf{z}) \times \beta^0 \\
&= 24 \times 26^5 + 25 \times 26^4 + 25 \times 26^3 + 24 \times 26^2 + 23 \times 26^1 + 25 \times 26^0 \\
&= 297033671.
\end{aligned}$$

Clearly, as β and k grow, the integer $z(S)$ can become arbitrarily large. Therefore, this value can be reduced modulo a prime number p to restrict it to the range $[0, p-1]$. This defines:

$$r(S) = z(S) \bmod p = \left(\sum_{i=1}^k \text{index}(s_i) \times \beta^{k-i} \right) \bmod p.$$

- **Using this mapping to enable exact pattern matching:** The above mapping enables a simple semi-numerical method for comparing substrings, replacing character-by-character comparison of the naive exact pattern matching algorithm with comparisons between two integers modulo p . In other words, given a text string $\text{txt}[1 \dots n]$ and a pattern string $\text{pat}[1 \dots m]$ over some specified alphabet Σ , this integer mapping modulo p can be used to determine all positions in the text where the pattern occurs as a substring.

The overarching idea is as follows. First, the value $r(\text{pat}[1 \dots m])$ is precomputed for the full pattern string. Then, for each position $1 \leq j \leq n - m + 1$, the value $r(\text{txt}[j \dots j + m - 1])$ is computed for the corresponding length- m substring of the text, effectively sliding a window of length m across txt . In any iteration, if $r(\text{txt}[j \dots j + m - 1]) \neq r(\text{pat}[1 \dots m])$, then this implies $\text{txt}[j \dots j + m - 1] \neq \text{pat}[1 \dots m]$. However, when the values are equal, only then a direct character-by-character verification must be performed to confirm the match. (This is necessary because all comparisons are performed modulo p , so distinct substrings may produce the same r value.)

- **Short example to illustrate the method** Consider a short alphabet $\Sigma = \{a, b, c, d, e\}$, with $\text{index}(a) = 0, \text{index}(b) = 1, \dots, \text{index}(e) = 4$. For this example's sake, assume the prime modulus used was $p = 11$. Consider the pattern $\text{pat} = \text{abe}$. Using $\beta = |\Sigma| = 5$, the pattern is interpreted as a base-5 number modulo 11:

$$\begin{aligned} r(\text{abe}) &= 0 \times 5^2 + 1 \times 5^1 + 4 \times 5^0 \\ &= 5 + 4 = 9 \\ &\equiv 9 \pmod{11}. \end{aligned}$$

Now consider a text $\text{txt} = \text{cabea}$. Sliding along the text with a window size $m = 3$, the length-3 substrings encountered will be cab , abe , bea , and their computed r values will be:

$$\begin{aligned} r(\text{cab}) &= 2 \times 5^2 + 0 \times 5^1 + 1 \times 5^0 = 51 \equiv 7 \pmod{11}, \\ r(\text{abe}) &= 0 \times 5^2 + 1 \times 5^1 + 4 \times 5^0 = 9 \equiv 9 \pmod{11}, \\ r(\text{bea}) &= 1 \times 5^2 + 4 \times 5^1 + 0 \times 5^0 = 45 \equiv 1 \pmod{11}. \end{aligned}$$

Thus, comparing just the r values, a (potential) match is detected at the second position of this text and the direct verification confirms the occurrence.

- **Key optimisation:** An important observation that, in general, $r(\text{txt}[j \dots j + m - 1])$ can be efficiently **updated** from the previously computed $r(\text{txt}[j - 1 \dots j + m - 2])$ without having to generate it using the full m -gram substring of that iteration. As part of this assignment, by analysing the computations of the r values in two successive iterations, you are expected to derive this update.

Your assignment task: Implementing the semi-numerical pattern matching approach described above, you must write an efficient Python program to find all occurrences of $\text{pat}[1 \dots m]$ in $\text{txt}[1 \dots n]$. In this task, assume that these input strings to your program are drawn from the full plain ASCII alphabet Σ with codes in the range $[0, 127]$ (so $|\Sigma| = \beta = 128$).

As part of this task, your program must first generate a prime modulus p . For this, your program must *randomly* sample candidate integers between $[2^{t-1}, 2^t - 1]$, where

$t = \max(32, m)$, and test each candidate using the Miller–Rabin primality test (covered in Week 6) until a random prime p is found. You may use Python’s `random` module to generate candidate numbers.

You will also have to implement modular exponentiation using repeated squaring (covered in Week 6) and cannot use Python’s built-in modular exponentiation.

Otherwise, you are allowed to use basic built-in integer arithmetic operations (+, −, ×, %). (Among other things, this means you do not have to implement Karatsuba multiplication for this task.)

Your program should have the following high level structure:

```

Test random  $t$ -bit candidates for primality until a prime  $p$  is found

Compute  $r(\text{pat}[1..m])$ 

For each  $j$  from 1 to  $n - m + 1$ :
    Compute  $r(\text{txt}[j..j + m - 1])$  efficiently

    If  $r(\text{pat}[1..m]) == r(\text{txt}[j..j + m - 1])$ :
        verify if  $\text{pat}[1..m] == \text{txt}[j..j + m - 1]$ 
        If equal output position  $j$ 

```

Strictly follow the following specification to address this question:

Program name: `a2q2.py`

Arguments to your program: Two filenames:

- (a) First containing `txt[1...n]` string (in a single line of the file).
- (b) Second containing `pat[1..m]` string (in a single line of the file).

(You are free to assume that the text and pattern strings are non-empty and that pattern is at most as long as the text.)

Command line usage of your script:

```
python a2q2.py <text filename> <pattern filename>
```

Do not hard-code the input filenames within your program. Ensure your program can be executed from the terminal (command line) by supplying any pair of filenames (containing text and pattern strings) as command-line arguments. Include sufficient inline comments explaining each logical block of your code. Code that lacks adequate comments will require a face-to-face interview to verify your understanding before it can be assessed.

Output file name: `output_a2q2.txt`

- Each position where `pat` matches the `txt` must be reported on a new line using 1-based indexing (even though your program will compute them in 0-based indexing). If no matches are found, the output file will be empty.

- **Example:** Given `txt = b a b b a a b a b a a b a b a a b a b a b` and `pat = a a b a b a a b a b`, the output file will contain:

```

5
10

```

No further test cases will be provided. You must develop your own test cases.

PDF Report: `report_a2q2.pdf`

- Explain how $r(\texttt{txt}[j \dots j+m-1])$ was updated from previous $r(\texttt{txt}[j-1 \dots j+m-2])$.

(No run log necessary for this task.)

--o0o--

END

--o0o--